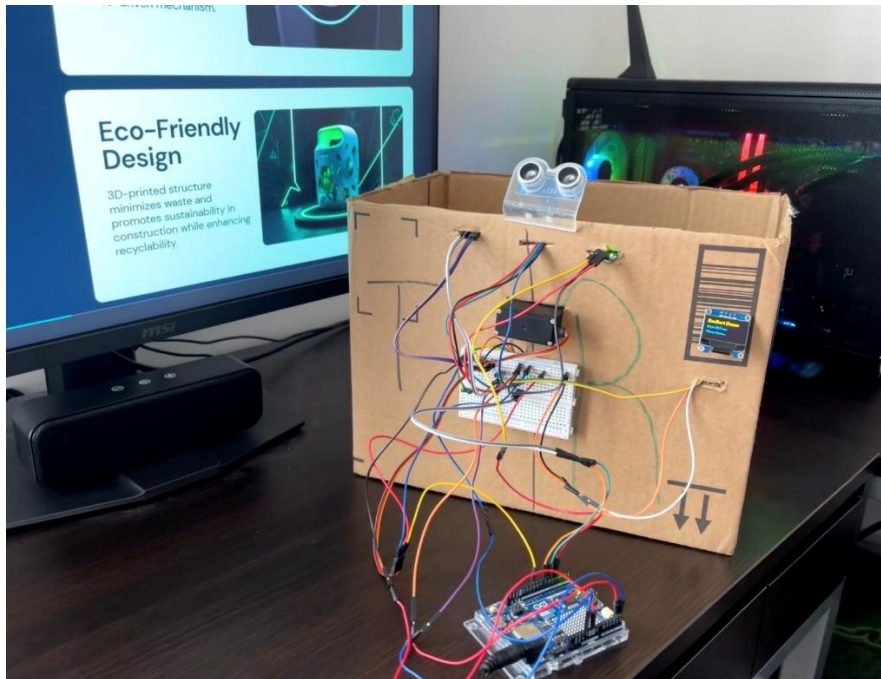


EcoSort Bin: Smart Automated Waste-Sorting System

“A Trash Bin with a College Degree”



Prepared by: Yusuf Guenena

Course: Embedded Systems Design

Instructor: Dr. Claude Gauthier

Wayne State University

Date: 12/12/2025

Abstract

EcoSort is a low-cost automated waste-sorting system built with a microcontroller, lightweight sensors, and a servo-driven sorting platform. This project aims to reduce contamination of recyclables in apartments, classrooms, and public places by allowing the bin itself to identify the type of waste thrown into it, rather than requiring users to provide this information. EcoSort uses a standard set of sensors to identify the waste item's characteristics using load cell weights, ultrasonic ranges, infrared reflectivity, and color. Based on the weight, color, and surface reflectivity on the intake platform, EcoSort classifies the contents of the bin as trash (paper/cardboard) or recyclable (plastic bottles/containers) via an internal, servo-controlled seesaw mechanism that directs the waste item to its appropriate compartment. The results show that this Arduino-based project can be implemented on a bigger scale to provide sorting bins throughout college campuses and public spaces. Also, when multi-sensor fusion is applied through simple embedded processing, it can improve sorting accuracy without the need for computer vision or machine learning models.

1. Introduction

Recycling contamination continues to be a problem for institutions and communities alike. Most individuals do not have a good guess as to what bin they should use, especially with regard to small items that are utilized regularly, including bottles, cups, paper, and wrappers, which results in 25-35% of all materials placed in recycling bins ultimately being disposed of as landfill. The end result is a higher incidence of rejected loads and greater contributions of waste into landfills.

EcoSort is an innovative product designed to assist by providing an automatic classification system for common waste items to assist with correct bin location. EcoSort's autonomous classification system utilizes sensor technologies rather than reliance upon user judgement to classify the type of waste being placed in the bin and directs the waste into the respective bin for recycling via mechanical means. In addition to making it easier for all individuals to recycle correctly, EcoSort reduces the level of human error in the recycling process and increases the ability to accurately classify and separate waste materials in public settings. The project highlights the development of a smart bin utilizing low-cost sensors and Arduino-based systems to automatically classify and sort waste materials.

2. Marketing Analysis

Contamination by recycling has become a common problem for nearly every organization. Universities often have mixed waste in their dorms and student unions; coffee cups, paper and plastic are commonly disposed of in office buildings; hospitals and clinics depend on the separation of materials; restaurants and shopping malls generate an enormous amount of consumer packaging daily.

EcoSort provides an easy-to-use, affordable alternative for all those locations that experience problems related to contamination. The following are the primary reasons for the growth of EcoSort in the marketplace:

- Demand for Sustainability
- More effective sorting will reduce the amount of landfill waste that must be thrown away and help to meet our goals related to sustainability.
- Increased Savings
- Facilities will be able to decrease the amount of money spent on getting rejected bales of recyclable materials sent to landfills.
- Simple Installation

A cardboard box can be used as an enclosure to support the EcoSort system and has no complicated electronic parts, sensors, etc., and can easily be expanded or updated.

Educating Others:

Universities can use EcoSort as a platform to educate students about environment-related topics while allowing them to continue their current conduct without the need to change their daily routines.

Overall, the growth of EcoSort continues as the market recognizes that embedded and automated solutions provide both service to users as well as support for waste management systems.

3. Implementation

3.1. Background Materials

EcoSort uses the following primary components:

- Arduino Uno R4 (main controller)
- Load cell + HX711 amplifier (weight measurement)
- Ultrasonic sensor (object height + presence detection)
- IR reflective sensor (material reflectivity)
- TCS34725 color sensor (RGB detection)
- MG996R servo motor (sorting actuation)
- OLED display (system messages)
- Cardboard enclosure (prototype housing)

Each component was selected because it is inexpensive, reliable, and easy to integrate into an Arduino-based system.

3.2. Theory of Design

EcoSort uses rule-based classification based on weight + color + reflectivity (+ ultrasonic presence/height):

- Lightweight (between 10-12g) - Examples include shredded paper and small pieces of cardboard, categorized as rubbish
- Medium weight (between 20-30g) - Examples include all types of plastic bottles and pieces of mint, categorized as recyclable
- Green items - an example of green items is mint container, categorized as recyclable
- Highly Reflective - an example of a plastic bottle that is collected and categorized as recyclable

After classification, the intake platform (which uses a servo to rotate) rotates left or right to deposit the item in the appropriate compartment.

A cardboard box is a relatively simple yet efficient enclosure in which to test & create, using internal dividers to provide the waste compartments.

3.3. Operating Procedure

1. Users place an item on the intake platform.
2. Ultrasonic sensor detects object presence.
3. Load cell measures object weight.
4. Color sensor reads RGB values.
5. IR sensor determines surface reflectivity.
6. Arduino processes all readings and classifies the item.
7. OLED displays “Recyclable” or “Trash.”

8. Servo rotates:
 - Left → Recyclable
 - Right → Trash
9. Servo returns to center for the next item.

3.4. Analysis, Results, and Discussion

Four items were used for testing:

Item	Weight	Reflective	Color	Classification
Paper	10 g	No	White	Trash
Cardboard	12 g	No	Brown	Trash
Mint Container	22 g	No	Green	Recyclable
Plastic Bottle	27 g	Yes	Light Color	Recyclable

Sorting Precision

40 trials (10 per object type) were conducted. Here's how each trial scored. (9 trials were excluded due to mechanical jamming and are discussed in Section 9.2 Problems Faced)

Paper: 10 times correct

Cardboard: 9 times correct

Mint Container: 10 times correct

Plastic Bottle: 10 times Correct

Overall Accuracy: 97.5%

Issues in sorting accuracy stemmed from environmental conditions. The only sorting error was that of the cardboard, due to it being under unusual lighting conditions; the color brightness was not read correctly.

Strengths of the System

- Rapid classification of items
- The OLED display clearly indicates the status of sorting results (Correct/Incorrect).
- The system's lightweight frame makes it easy to perform demonstrations.
- Its cardboard covering makes it easy to assemble and transport the system.

Weaknesses of the System

- A small amount of flexing in the cardboard will impact the stability of the load cell.
- Color reading errors may occur due to bright sunlight impacting the color spectrum.
- The system can only be used to sort small items with a maximum weight of 200 grams or less.

3.5. Estimated Cost of System

Component	Description/Role	Qty	Unit Cost (USD \$)
Arduino R4 WIFI	Main microcontroller, reads sensors and drives servo /UI	1	29.00
Load Cell (5 Kg)	Measures weight of the item placed on the platform	1	6.99
HX711 Amplifier Module	Amplifies and digitizes load cell signal for Arduino	1	0.00
Servo Motor (MG996R)	Rotates the seesaw platform left/right	1	6.99
OLED Screen (with I2C interface)	Displays prompts and classification results	1	6.99
Color Sensor (TCS34725)	Detects color of waste items for classification	1	3.53
Ultrasonic Sensor (HC-SR04)	Detects object presence / distance	1	3.50
IR Reflective Sensor (TCRT5000)	Detects object alignment / material type for classification	1	0.87
Cardboard enclosure	The main outer shell	1	0.00
Breadboard, wiring, 3D printed parts	-	1	~10.00
Total		~10	~68 – 75

Cost Reduction Options

- Replace MG996R with SG90 (if torque requirement reduced)
- Use a smaller OLED (or an LCD screen instead)
- Bulk-order sensors

3.6. Conclusion

EcoSort has shown that you can create a dependable waste sorting solution by using inexpensive, low-tech sensors paired with simple mechanics and straightforward rules or algorithms for classification within an embedded microcontroller. Despite being housed in a cardboard box, the accuracy of recycled material sorting is improved by this prototype's consistency.

4. Go-to-Market Strategy

EcoSort is not intended as a final consumer product but as proof-of-concept that can be expanded into a full smart bin system.

Initial Targets

- Universities
- Office buildings
- Cafeterias
- Public libraries

Advantages

- Low-cost implementation
- Easy installation
- Educational value
- Demonstrates sustainability commitment

Future Product Version

- Plastic or metal enclosure
- Larger intake platform
- Battery-powered operation
- IoT dashboard for waste analytics

EcoSort could serve as a starting point for campuses seeking greener infrastructure without high investment.

5. System Block Diagrams

5.1. High-Level system Block diagram

As soon as users set a product on EcoSort's platform, the system starts analyzing it immediately with multiple sensors working together. To help determine if an object should be classed as recyclable or as general waste, EcoSort analyses the item's weight using the load cell and HX711 amplifier, identifies its color and brightness using the TCS34725 color sensor, detects whether the product has a metallic shine using the IR reflective sensor and utilizes the ultrasonic sensor to detect the presence of the product as well as estimate its height. The results from these various sensors are combined within the Arduino Uno, and the system makes a decision about the product's classification by combining the inputs from the various sensors. After the decision has been made about classification, the servo motor (MG996R) rotates the seesaw plate to direct the product towards the correct bin for disposal or recycling. The system provides prompt and clear feedback to users via the OLED display and buzzer throughout the entire process by prompting users to place an item, indicating when the system is currently measuring, and

confirming the final decision regarding item classification. This complete system allows users to experience an automated sorting process that takes away the guesswork from how to recycle and makes the act of recycling as simple as possible.

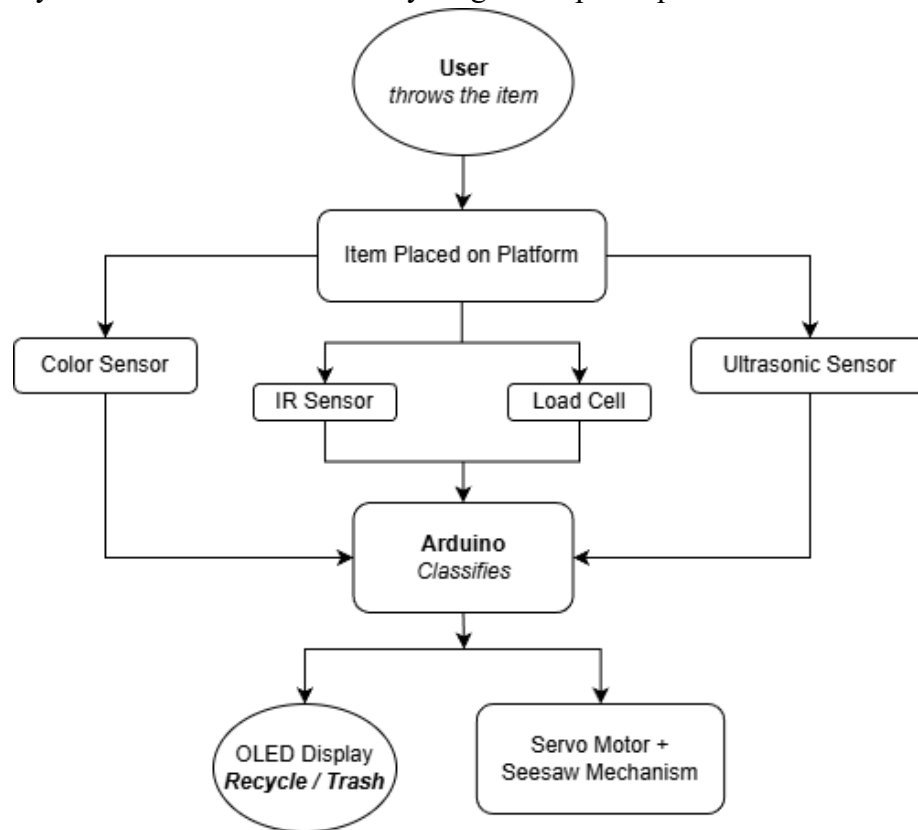


Figure 1. System Block Diagram

5.2. Hardware Block Diagram

The hardware of the system is based on an Arduino Uno, which acts as the central computing and coordination device. All other sensors/actuators/interface components connect to the Arduino through various communication methods, including:

- **HX711 load cell** amplifier utilizing two digital pins, DT and SCK, to give high-resolution measurements of weight.
- **TCS34725 color sensor** communicating via an I²C bus, SDA/SCL, sharing this connection with the OLED display connected to the same bus.
- **HW-870 IR reflective sensor** connecting via a digital input pin that detects metal based on the percentage of reflectivity.
- **HC-SR04 ultrasonic** rangefinder using two digital pins, Trigger and Echo, to determine the height/distance of an object.
- **MG996R servo motor** controlled via a PWM-enabled digital pin to control the angle of rotation accurately.
- **OLED display** using I²C communication connected to the Arduino to retrieve status messages.

- 5 V regulated power supply used to power the Arduino and all sensors; the servo motor is powered from a separate 5-6 V supply to avoid brownout conditions due to excessive current draw.

Together, these components form an integrated sensor-fusion architecture where the Arduino collects data from multiple sensing channels, processes classification logic, and actuates the sorting mechanism accordingly.

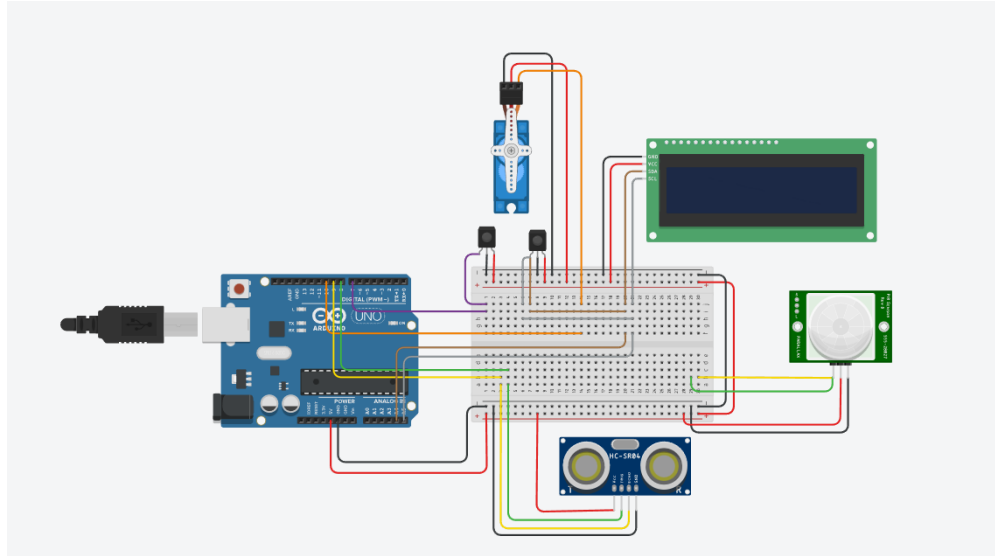


Figure 2. Hardware Architecture Block Diagram of EcoSort

Certain components (OLED display, load cell, and color sensor) were illustrated using alternative symbols because the exact device models were not available in the diagramming software.

6. Mechanical Design

6.1. Overview

By design, the EcoSort has a streamlined, functional mechanical configuration that provides accurate detection and consistent sorting processes. EcoSort's mechanical configuration is focused on a flat platform for item intake, while the EcoSort is equipped with an electrically actuated pivot that allows the items to be turned around into one of two separate compartments. Given their limited time and scope to develop a commercially available product, the EcoSort team focused their resources and efforts on developing a strong platform and sorting mechanism rather than developing a complete commercial-style bin. For ease of demonstration purposes, both compartments were created in one individual plastic container, with the separating divider being removable.

6.2. Intake Platform and Sorting Mechanism

The intake platform is designed as a seesaw-style surface driven by an MG996R high-torque servo. The conceptual design can be seen in Figure 3. When an item is placed and classified:

- Servo rotates left → item falls into the recyclable compartment
- Servo rotates right → item falls into the trash compartment

Internal supports guide items cleanly into their respective compartments to reduce jamming and ensure consistent operation.

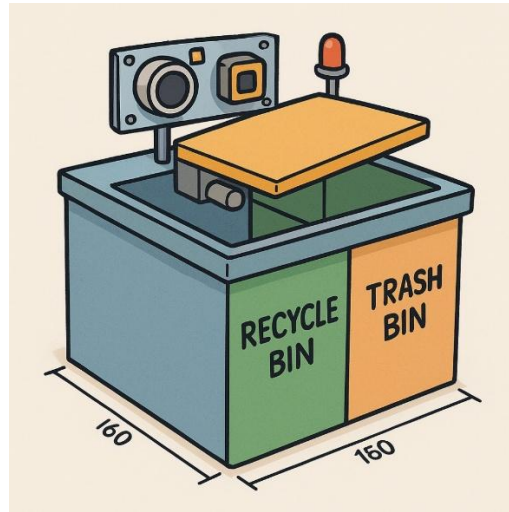


Figure 3. Shows a sketch of the prototype of the mechanism

6.3. 3D-Printed Enclosure and Iterative Design Process

Because this mechanism is entirely 3D-printed, the design process involves multiple stages of iteration and refinement. From the beginning, we aimed to create an enclosure that could securely house the load cell while also allowing the entire platform to move freely during sorting. Our initial 3D model, shown in the figures below, went through three major iterations as we adjusted dimensions, mounting points, and clearances to ensure proper alignment and sensor fit. The final design uses a two-plate structure consisting of a top plate and a bottom plate. This layout was chosen so the load cell could be sandwiched between the plates, allowing it to accurately measure the weight of any item placed on the platform. As illustrated in the figures, the top plate includes mounting holes that align with matching protrusions on the bottom plate. These interlocking features keep the platform stable and properly aligned as it moves. Both plates include a hollow rectangular cutout in the center, which serves as the mounting cavity for the load cell. Additionally, the bottom plate includes a semi-circular opening designed to accommodate the drive shaft.

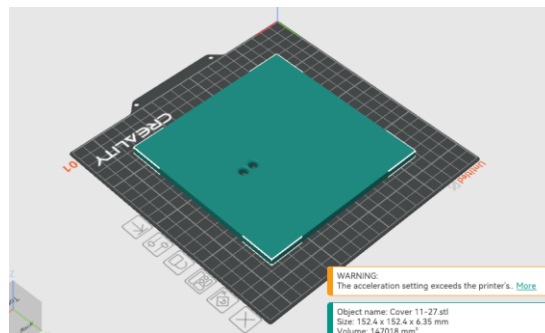


Figure 4. Shows 3D design of the top plate

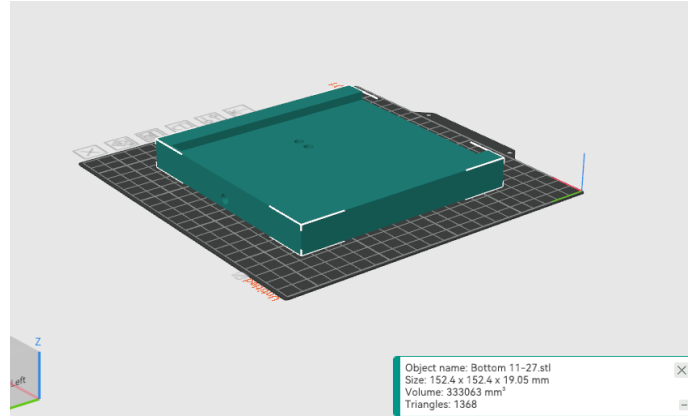


Figure 5. Shows 3D design of the bottom plate

6.4. Drive Shaft and Platform Support

The drive shaft, which is a D-type steel rod, connects the servo to the platform and enables it to rotate left or right during sorting. The shaft attaches to the servo through a coupler, shown in Figure 6, ensuring a secure and reliable connection between the actuation system and the platform.

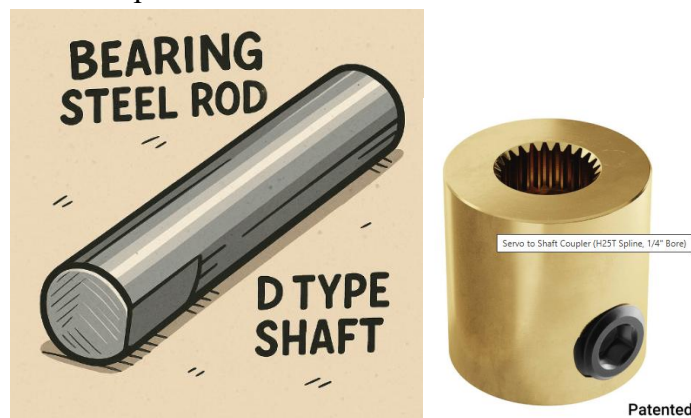


Figure 6. Shows the D type shaft and the coupler connecting to the servo

6.5. Sensor Installation and Calibration

The system's sensing performance depends heavily on proper physical mounting and calibration of each sensor. Each component was positioned to ensure consistent measurements and minimize noise or misalignment.

- **Load Cell + HX711**

Installed between the top and bottom plates using recessed mounting screws. Calibration included taring the empty platform, applying known reference weights, and generating a linear conversion factor for stable gram-level readings. The way it needs to be installed matter greatly for accurate reading. As shown in figure 7, the load cell would need to be installed between the two platforms but have only one point of contact with each platform (spacers). To calibrate this load cell, we will need to run a code and use an item with a known weight to calculate

the calibration factor. The calibration factor is important to help establish a more accurate and calibrated reading from the load cell.

$$\text{Calibration factor} = (\text{reading})/(\text{known weight})$$

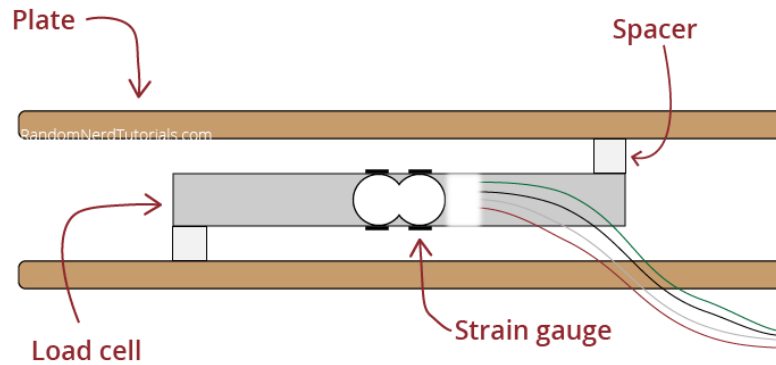


Figure 7. Load cell installation guide

- **Color Sensor (TCS34725)**

Mounted beneath a viewing window in the intake plate with a fixed 2–3 cm sensing distance. Calibration used white, black, and cardboard reference samples, normalizing RGB values, and checking stability with the built-in LED.

- **IR Reflective Sensor**

Placed at an angled position facing the underside of the intake platform. Calibration involved tuning the potentiometer until aluminum objects reliably produced a high reflectivity signal while plastic and paper remained below the threshold.

- **Ultrasonic Sensor (HC-SR04)**

Mounted at the front of the platform to detect object placement and height. Baseline measurements were established for an empty platform, with thresholds configured for tall vs. flat objects.

Collectively, the mounting strategy and calibration process provided consistent, repeatable sensor fusion performance during classification tests.

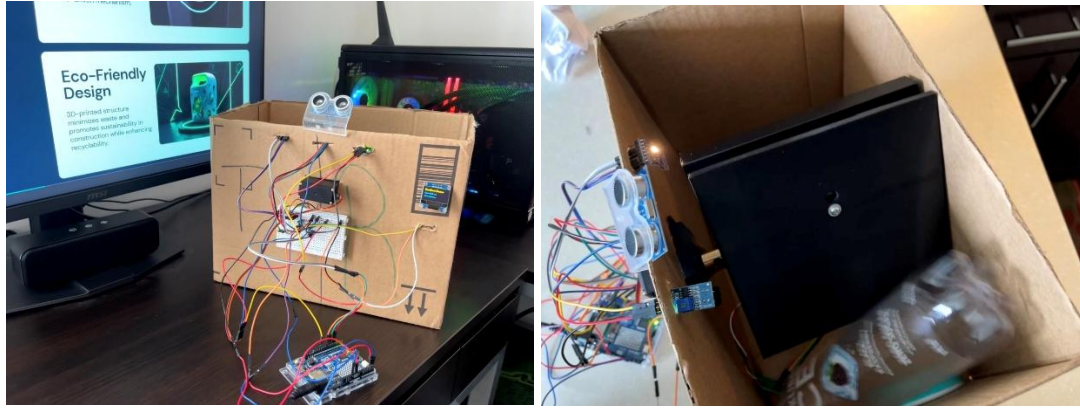


Figure 8. Final prototype with all sensors installed

7. Future Improvements

To make EcoSort more advanced and closer to commercial systems:

1. Add camera + machine learning classification.
2. Add multiple sensors (metal detector).
3. Add Wi-Fi logging to track recycling data.
4. Replace servo with stronger mechanism.
5. Add battery power system.
6. Improve enclosure design.

8. References

- [1] U.S. EPA, “National Overview: Facts and Figures on Materials, Waste and Recycling,” 2024.
- [2] Recycle Across America, “Recycling Contamination Facts,” 2023.
- [3] NWRA, “Impact of Contamination in Recycling Streams,” 2023.
- [4] Prasad, K., Chandrasekar, A., “A Smart Waste Management System Using Sensor-Based Technology,” *IEEE Sensors Journal*, 2022.
- [5] Khan, M., Arif, P., “Low-Cost Smart Bin Using Weight and Color Sensors,” *IEEE IoT Conference*, 2021.

9. Appendix

9.1. Alternative Designs Considered

- A conveyor belt system
- A drop-door trap system
- A vision-based sorting system using a camera
- A vertical chute with rotating gate

The seesaw mechanism was chosen because it is simple, reliable, and easy to build inside a cardboard box.

9.2. Problems Faced

- Load cell drift due to cardboard flex
- Servo stalling before switching to MG996R

- Color sensor affected by overhead lighting
- Ultrasonic noise near edges of enclosure

9.3. Distribution of Work

Since this is a solo project:

Programming — Yusuf

Wiring & electronics — Yusuf

Mechanical design — Yusuf

Testing & calibration — Yusuf

Documentation — Yusuf

9.4. Program Listing

```

1.  /*
2.   EcoSort - Smart Automated Waste Sorting Bin
3.   Sensors:
4.     - Load Cell + HX711 (weight)
5.     - HC-SR04 Ultrasonic (presence / height proxy)
6.     - TCS34725 Color Sensor (RGB + Clear)
7.     - IR Reflective Sensor (digital threshold)
8.   Outputs:
9.     - MG996R Servo (seesaw)
10.    - SSD1306 OLED (status)
11.    - Optional buzzer
12.
13.   Tested items:
14.     - Paper 10g, white, NOT reflective -> TRASH
15.     - Cardboard 12g, brown, NOT reflective -> TRASH
16.     - Mint container 22g, green, NOT reflective -> RECYCLE
17.     - Plastic bottle 27g, light color, REFLECTIVE -> RECYCLE
18. */
19.
20. #include <Arduino.h>
21. #include <Wire.h>
22. #include <Servo.h>
23. #include "HX711.h"
24. #include <Adafruit_TCS34725.h>
25. #include <Adafruit_GFX.h>
26. #include <Adafruit_SSD1306.h>
27.
28. // ----- PINS -----
29. const int HX_DOUT_PIN = 3;
30. const int HX_SCK_PIN  = 2;
31.
32. const int IR_REFLECT_PIN = 4;    // digital output from HW-
    870/TCRT5000 module
33.

```

```

34.const int SERVO_PIN = 5;
35.const int BUZZER_PIN = 6;           // optional
36.
37.const int US_TRIG_PIN = 8;         // HC-SR04 TRIG
38.const int US_ECHO_PIN = 9;         // HC-SR04 ECHO
39.
40.// I2C: SDA/SCL are on A4/A5 on UNO-style boards (UNO R4 uses same
   header pins)
41.
42.// ----- OLED -----
43.#define SCREEN_WIDTH 128
44.#define SCREEN_HEIGHT 64
45.#define OLED_RESET -1
46.Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
   OLED_RESET);
47.
48.// ----- COLOR SENSOR -----
49.Adafruit_TCS34725 tcs(
50.  TCS34725_INTEGRATIONTIME_50MS,
51.  TCS34725_GAIN_4X
52.);
53.
54.// ----- SCALE -----
55.HX711 scale;
56.
57.// IMPORTANT: Set this from your calibration
58.// Run calibration with a known weight and adjust until grams look
   right.
59.float CALIBRATION_FACTOR = 2280.0f;
60.
61.// ----- SERVO -----
62.Servo gateServo;
63.
64.// Servo angles (adjust to match your mechanism)
65.const int SERVO_RECYCLE = 0;  // left
66.const int SERVO_CENTER = 90; // neutral
67.const int SERVO_TRASH = 180; // right
68.
69.// ----- TIMING -----
70.const unsigned long SETTLE_MS = 700; // wait after detecting
   item
71.const unsigned long ACTUATE_MS = 900; // time to keep tilted so
   item drops
72.const unsigned long RETURN_MS = 300; // time after returning
   to center

```

```

73.const unsigned long COOLDOWN_MS    = 800;    // avoid double-trigger
74.
75.// ----- THRESHOLDS (tune if needed) -----
    ---
76.// Presence detection thresholds
77.const float WEIGHT_PRESENT_G    = 5.0f;        // if weight > this,
    assume item present
78.const float HEIGHT_PRESENT_CM    = 15.0f;        // if distance drops below
    this, item present (depends on mounting)
79.                                                // You will calibrate this
    based on your empty distance.
80.
81.// Classification thresholds
82.const float LIGHT_TRASH_G        = 15.0f;        // < 15g -> TRASH
    (paper/cardboard)
83.const float RECYCLE_MAX_G        = 60.0f;        // safe upper bound for
    these demos
84.
85.// Green detection for mint container (ratio)
86.const float GREEN_RATIO_MIN      = 0.30f;
87.
88.// Optional: "tall object" heuristic (plastic bottle)
89.const float TALL_OBJECT_CM       = 10.0f;        // if estimated height >=
    this -> likely bottle (depends on setup)
90.
91.// ----- TYPES -----
92.enum ItemClass {
93.    CLASS_TRASH,
94.    CLASS_RECYCLE,
95.    CLASS_UNKNOWN
96.};
97.
98.struct Measurement {
99.    float weight_g;
100.    float distance_cm; // ultrasonic distance to object (not true
    height unless you compute from empty distance)
101.    bool reflective;
102.    uint16_t r, g, b, c;
103.    float rn, gn, bn; // normalized ratios
104.};
105.
106.// ----- STATE MACHINE -----
107.enum State {
108.    STATE_IDLE,
109.    STATE_DETECTED,

```



```

110.     STATE_SETTLE,
111.     STATE_MEASURE,
112.     STATE_DECIDE,
113.     STATE_ACTUATE,
114.     STATE_RETURN,
115.     STATE_COOLDOWN
116. };
117.
118. State state = STATE_IDLE;
119. unsigned long stateStartMs = 0;
120.
121. // Used to convert ultrasonic distance to "height proxy"
122. float emptyDistanceCm = 0.0f; // measured at startup
123.
124. // ----- HELPERS -----
125. void beep(int ms, int times=1) {
126.     if (BUZZER_PIN < 0) return;
127.     for (int i=0; i<times; i++) {
128.         digitalWrite(BUZZER_PIN, HIGH);
129.         delay(ms);
130.         digitalWrite(BUZZER_PIN, LOW);
131.         delay(60);
132.     }
133. }
134.
135. void oledPrint(const String &l1, const String &l2="", const String
    &l3="", const String &l4="") {
136.     display.clearDisplay();
137.     display.setCursor(0, 0);
138.     display.print(l1);
139.     display.setCursor(0, 16);
140.     display.print(l2);
141.     display.setCursor(0, 32);
142.     display.print(l3);
143.     display.setCursor(0, 48);
144.     display.print(l4);
145.     display.display();
146. }
147.
148. float readUltrasonicCm() {
149.     // HC-SR04: trigger 10us pulse, measure echo duration
150.     digitalWrite(US_TRIG_PIN, LOW);
151.     delayMicroseconds(2);
152.     digitalWrite(US_TRIG_PIN, HIGH);
153.     delayMicroseconds(10);

```

```

154.     digitalWrite(US_TRIG_PIN, LOW);
155.
156.     unsigned long duration = pulseIn(US_ECHO_PIN, HIGH, 30000UL); //
    30ms timeout (~5m)
157.     if (duration == 0) return 999.0f; // no echo
158.     // distance cm ~ duration/58
159.     return (float)duration / 58.0f;
160. }
161.
162. float readWeightGrams(int samples=15) {
163.     // Average multiple scale readings to reduce noise
164.     float sum = 0.0f;
165.     for (int i=0; i<samples; i++) {
166.         sum += scale.get_units(1);
167.     }
168.     return sum / (float)samples;
169. }
170.
171. void readColor(Measurement &m, int samples=5) {
172.     uint32_t r_acc=0, g_acc=0, b_acc=0, c_acc=0;
173.     for (int i=0; i<samples; i++) {
174.         uint16_t r,g,b,c;
175.         tcs.getRawData(&r,&g,&b,&c);
176.         r_acc += r; g_acc += g; b_acc += b; c_acc += c;
177.         delay(10);
178.     }
179.     m.r = (uint16_t)(r_acc / samples);
180.     m.g = (uint16_t)(g_acc / samples);
181.     m.b = (uint16_t)(b_acc / samples);
182.     m.c = (uint16_t)(c_acc / samples);
183.
184.     float C = (m.c == 0) ? 1.0f : (float)m.c;
185.     m.rn = (float)m.r / C;
186.     m.gn = (float)m.g / C;
187.     m.bn = (float)m.b / C;
188. }
189.
190. bool readReflective() {
191.     // Most IR modules: LOW or HIGH depending on threshold.
192.     // If your module behavior is opposite, flip this logic.
193.     int v = digitalRead(IR_REFLECT_PIN);
194.     // assume HIGH = reflective detected (common on some boards),
    adjust if needed
195.     return (v == HIGH);
196. }

```

```

197.
198. ItemClass classify(const Measurement &m) {
199.     // Convert distance to a "height proxy" using emptyDistanceCm if
    you want:
200.     // heightProxy = emptyDistance - currentDistance
201.     float heightProxy = (emptyDistanceCm > 0 && m.distance_cm < 900)
    ? (emptyDistanceCm - m.distance_cm) : 0.0f;
202.
203.     // Rule 1: very light -> TRASH (paper/cardboard)
204.     if (m.weight_g < LIGHT_TRASH_G) {
205.         return CLASS_TRASH;
206.     }
207.
208.     // Rule 2: demo recycle range
209.     if (m.weight_g >= LIGHT_TRASH_G && m.weight_g <= RECYCLE_MAX_G)
    {
210.         // Mint container: green dominant
211.         if (m.gn > GREEN_RATIO_MIN) {
212.             return CLASS_RECYCLE;
213.         }
214.
215.         // Plastic bottle: reflective
216.         if (m.reflective) {
217.             return CLASS_RECYCLE;
218.         }
219.
220.         // Optional: tall object -> likely bottle
221.         if (heightProxy >= TALL_OBJECT_CM) {
222.             return CLASS_RECYCLE;
223.         }
224.
225.         // Otherwise unknown -> TRASH (safe)
226.         return CLASS_TRASH;
227.     }
228.
229.     // Heavy unknown -> TRASH by default
230.     return CLASS_TRASH;
231. }
232.
233. void go(State s) {
234.     state = s;
235.     stateStartMs = millis();
236. }
237.
238. // ----- SETUP -----

```

```
239. void setup() {
240.     Serial.begin(115200);
241.     delay(400);
242.
243.     pinMode(IR_REFLECT_PIN, INPUT);
244.     pinMode(BUZZER_PIN, OUTPUT);
245.     digitalWrite(BUZZER_PIN, LOW);
246.
247.     pinMode(US_TRIG_PIN, OUTPUT);
248.     pinMode(US_ECHO_PIN, INPUT);
249.
250.     // OLED
251.     if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
252.         // If OLED not found, still continue without it
253.     } else {
254.         display.clearDisplay();
255.         display.setTextSize(1);
256.         display.setTextColor(SSD1306_WHITE);
257.         oledPrint("EcoSort Booting...");
258.     }
259.
260.     // Servo
261.     gateServo.attach(SERVO_PIN);
262.     gateServo.write(SERVO_CENTER);
263.
264.     // Color sensor
265.     bool tcs_ok = tcs.begin();
266.     if (!tcs_ok) {
267.         Serial.println("ERROR: TCS34725 not found.");
268.     } else {
269.         Serial.println("TCS34725 OK");
270.     }
271.
272.     // HX711
273.     scale.begin(HX_DOUT_PIN, HX_SCK_PIN);
274.     scale.set_scale(CALIBRATION_FACTOR);
275.     scale.tare(); // zero the scale
276.     Serial.println("Scale tared.");
277.
278.     // Measure empty ultrasonic distance (baseline)
279.     // Take multiple samples to reduce noise
280.     float sum=0.0f;
281.     int n=10;
282.     for (int i=0; i<n; i++) {
283.         float d = readUltrasonicCm();
```

```

284.     sum += d;
285.     delay(50);
286. }
287. emptyDistanceCm = sum / n;
288. Serial.print("Ultrasonic empty baseline (cm): ");
289. Serial.println(emptyDistanceCm);
290.
291. oledPrint("EcoSort Ready", "Place Item...");
292. beep(80, 2);
293.
294. go(STATE_IDLE);
295. }
296.
297. // ----- LOOP -----
298. void loop() {
299.     unsigned long now = millis();
300.
301.     // quick reads for detection
302.     float w_quick = readWeightGrams(5);
303.     float d_quick = readUltrasonicCm();
304.
305.     bool presentByWeight = (w_quick > WEIGHT_PRESENT_G);
306.     bool presentByUS = (d_quick < HEIGHT_PRESENT_CM); // depends on
your mounting
307.
308.     switch (state) {
309.
310.         case STATE_IDLE:
311.             gateServo.write(SERVO_CENTER);
312.             oledPrint("EcoSort Ready", "Place Item...");
313.             // detect item
314.             if (presentByWeight || presentByUS) {
315.                 go(STATE_DETECTED);
316.             }
317.             break;
318.
319.         case STATE_DETECTED:
320.             oledPrint("Item Detected", "Stabilizing...");
321.             beep(60, 1);
322.             go(STATE_SETTLE);
323.             break;
324.
325.         case STATE_SETTLE:
326.             if (now - stateStartMs >= SETTLE_MS) {
327.                 go(STATE_MEASURE);

```

```

328.     }
329.     break;
330.
331.     case STATE_MEASURE: {
332.         Measurement m{};
333.         m.weight_g = readWeightGrams(15);
334.         m.distance_cm = readUltrasonicCm();
335.         m.reflective = readReflective();
336.         readColor(m);
337.
338.         // show measurements
339.         String l2 = "W: " + String(m.weight_g, 1) + "g";
340.         String l3 = "d: " + String(m.distance_cm, 1) + "cm";
341.         String l4 = String("G%: ") + String(m.gn, 2) + (m.reflective
            ? " Rfl:Y" : " Rfl:N");
342.         oledPrint("Measuring...", l2, l3, l4);
343.
344.         // log
345.         Serial.print("W(g)="); Serial.print(m.weight_g, 2);
346.         Serial.print(" d(cm)="); Serial.print(m.distance_cm, 2);
347.         Serial.print(" refl="); Serial.print(m.reflective ? "Y" :
            "N");
348.         Serial.print(" RGBc="); Serial.print(m.r);
            Serial.print(",");
349.         Serial.print(m.g); Serial.print(","); Serial.print(m.b);
350.         Serial.print(" c="); Serial.print(m.c);
351.         Serial.print(" rn/gn/bn="); Serial.print(m.rn, 3);
            Serial.print(",");
352.         Serial.print(m.gn, 3); Serial.print(",");
            Serial.println(m.bn, 3);
353.
354.         // store measurement locally by deciding immediately
355.         ItemClass cls = classify(m);
356.
357.         // decide target angle
358.         int targetAngle = SERVO_TRASH;
359.         String result = "TRASH";
360.         if (cls == CLASS_RECYCLE) {
361.             targetAngle = SERVO_RECYCLE;
362.             result = "RECYCLE";
363.         }
364.
365.         oledPrint("RESULT:", result,
366.             String("W: ") + String(m.weight_g, 1) + "g",

```

```
367.             m.reflective ? "Reflective: YES" : "Reflective:
    NO");
368.
369.         // actuate
370.         gateServo.write(targetAngle);
371.         beep(90, (cls == CLASS_RECYCLE) ? 2 : 1);
372.
373.         go(STATE_ACTUATE);
374.         break;
375.     }
376.
377.     case STATE_ACTUATE:
378.         if (now - stateStartMs >= ACTUATE_MS) {
379.             gateServo.write(SERVO_CENTER);
380.             go(STATE_RETURN);
381.         }
382.         break;
383.
384.     case STATE_RETURN:
385.         if (now - stateStartMs >= RETURN_MS) {
386.             go(STATE_COOLDOWN);
387.         }
388.         break;
389.
390.     case STATE_COOLDOWN:
391.         // Wait until item removed to avoid re-trigger
392.         oledPrint("Remove Item", "Resetting...");
393.         if ((now - stateStartMs >= COOLDOWN_MS) && !presentByWeight
    && !presentByUS) {
394.             go(STATE_IDLE);
395.         }
396.         break;
397.     }
398.
399.     delay(20); // small loop delay to reduce flicker / noise
400. }
401.
402.
```